

# Music and Multimedia Streaming over the Internet: Project 2026

Cristina Rottondi, Massimiliano Zanoni, Leonardo Severi

May 2, 2026

## Description

This project will give you the opportunity to experiment with the design and implementation of a networked, interactive system that explores the theme of *Human–Machine Interaction* (HMI). The final goal is to build a distributed application in which two software components, running on two different machines, exchange data over the network through the *Open Sound Control* (OSC) protocol and cooperate to produce a live audio/visual experience driven by human input.

The system is articulated in four conceptual components: sensing of human behaviour, mediation through Machine Learning, audio/visual rendering, and streaming of the resulting output to a web client as the performative dimension of the work. The two software components must be designed as distinct but interdependent entities, engaging in a meaningful exchange of data and behaviours.

Projects are expected to be framed within an artistic, performative or experiential context. The exchange of data via OSC must be treated as a core structural element of the work, not just a technical requirement, and the relationship between human agency and machine behaviour should inform the design of the interaction.

## Rules

1. The project is divided into a small number of steps. You are encouraged to address them in the presented order.
2. A part of the project is considered successfully developed if every student of the group is able to defend it and answer questions regarding the related theory (only if discussed during the course lectures).
3. Group size may range from one to four members. The overall complexity of the system must be proportionate to the number of group members. In particular, the variety of input features extracted, the number of trained models, and the richness of the audio/visual rendering should be chosen accordingly.
4. Each project must be conceived as a distributed system composed of two software entities communicating via OSC.
  - Groups of two or more members must deploy the two entities on two separate physical machines, communicating over the network.
  - Single-member projects may run the two entities as two independent processes on the same machine, simulating the networked interaction over `localhost`.

5. A specific technological stack is suggested for each component of the system, as detailed in the steps below. Alternative tools may be adopted where motivated by the project concept, provided that the resulting system can be reproduced without relying on external, third-party online services.
6. To develop your solution, you can use any tool or library compatible with the constraints above. However, support is only guaranteed on standard Linux installations with basic editing tools.

## Steps

### 1. Propose a project concept.

Before any implementation work begins, the group must submit a short abstract describing the artistic intention of the project, the kind of human input it intends to capture, and the audio/visual output it aims to produce. The abstract must be approved by the instructor; only then the group can move on to the implementation.

### 2. Implement the system.

The system is composed of four interdependent components. Their internal development order is left to the group; the components are described below as functional requirements rather than as a sequence.

- a. *Sensing (Machine 1)*. A first software entity is responsible for capturing human input and extracting a set of features from it. The input source is chosen by the group according to the project concept and may include, for example, mouse interaction, audio captured from a microphone, video captured from a camera, or higher-level descriptors such as hand or body landmarks. The number and variety of extracted features must be proportionate to the size of the group. Processing is the suggested environment for this component; Python is also acceptable, in particular when the chosen input source is more naturally handled there. The extracted features must be sent over the network as OSC messages.
- b. *Machine Learning (Machine 2)*. A second software entity, running on a separate machine, receives the feature stream via OSC and feeds it to one or more Machine Learning models trained by the group, using either classification or regression depending on the project concept. The models output a set of control parameters that are in turn forwarded as OSC messages. Wekinator is the suggested tool for this component. Equivalent alternatives are admissible, provided that the Machine Learning process remains a software entity clearly separated from sensing and rendering. The number of trained models and the dimensionality of their output must be proportionate to the size of the group.
- c. *Audio/visual rendering*. The control parameters produced by the Machine Learning stage are consumed by a rendering process that generates the audio and/or visual output of the system. The rendering process may live on either of the two machines, according to the design choices of the group; the OSC link between the Machine Learning stage and the rendering stage is therefore present in every configuration, either over the network or over `localhost`. Processing is again the suggested environment for this component, with alternatives admitted under the conditions stated in the rules. The expressive richness of the rendering must be proportionate to the size of the group.
- d. *Streaming*. The rendered output must be broadcast as a live stream to a web page that any client can open in a browser. OBS is required as the capture and broadcasting tool, and necessarily runs on the machine that hosts the rendering. The stream must be served by a self-hosted media server using a standard protocol such as WHIP or RTMP; `mediamtx` is the suggested server, but any equivalent self-hosted solution is acceptable. Reliance on

external streaming services is not allowed.

The two software entities must communicate via OSC as the only channel for the exchange of structural data between them. The design of the OSC messages (their addresses, payload, and rate) is a core part of the project and must reflect the conceptual relationship between the two entities.

### 3. **Prepare the live demo and the documentation.**

The project will be evaluated through a live demonstration or performance, in which the group operates the system in front of the instructor. In addition to the working prototype, the group must deliver:

- the complete source code of both software entities and any configuration needed to reproduce the setup;
- a brief written documentation, sufficient to understand the work without reading the code, covering the concept and artistic intention, the system architecture, the design of the OSC messages and the data flow between the two entities, and the technologies used.